हेलो! आपका स्वागत है **Unit 22: JavaScript Interview Mastery** की final study class में.

मैंने आपके workspace में, unit 22 js interview mastry folder के अंदर (directory names mapping matches) ye complete study files write kar di hain:

- Study Guide File: chapter22_interview_mastery.md
- Code Examples File: interview_examples.js
- Practice Exercises File: interview_practice.js

चलिए अब Variables scopes, dynamic this pointer structures, microtasks priorities, prototypes models, closures properties aur event loop sequences ko simple Hinglish explanation ke sath revise karte hain.

PART 1: CORE CONCEPTS & COMPARISONS

1. var vs let vs const

- **var:** function scoped, hoisted to undefined, re-declaration allowed.
- **let / const:** block scoped, hoisted into Temporal Dead Zone (TDZ), re-declaration throws error. const does not allow re-assignment.

---

2. == vs ===

- **==:** Abstract equality. type coercion (type conversion) karta hai comparison se pehle.
- **===:** Strict equality. bin conversion values aur types directly check karta hai.

---

3. null vs undefined

- **undefined:** variable creation completed, value assignment missing (default engine placeholder).
- **null:** developer manually sets value reference empty check.

---

4. Regular vs Arrow Functions

- **Regular:** dynamic this mapping depends on how/where the function is called. Can be called as constructor with new.
- **Arrow:** lexical this resolution (inherits surrounding parent block environment). Cannot be called with new.

---

5. Shallow vs Deep Copy

- **Shallow:** copies first level keys properties only. Nested properties remain linked by reference (Spread operator).
- **Deep:** recursively creates standalone clones of all nested levels (structuredClone() or JSON.parse(JSON.stringify())).

PART 2: ADVANCED JS REVISIONS

1. Prototype & Inheritance

JavaScript objects have a hidden link prototype property pointing to parent objects template, creating prototype chains ending in null.

Example

javascript

```
class Parent {
   constructor(name) { this.name = name; }
}
class Child extends Parent {
   constructor(name) { super(name); }
}
let c = new Child("Ravi");
console.log(c.name);
```

Output Ravi

---

2. Event Bubbling & Delegation

- **Bubbling:** default propagation travelling from child target upwards to parents.
- **Delegation:** single parent element listener handles dynamic children trigger events.

🧠 **QUICK REVISION SHEET**

- **Priority Rule:** Microtasks (Promises callbacks) execute before Macrotasks (setTimeout callbacks).
- **Hoisting Rule:** Functions declarations are hoisted fully; var as undefined; let/const inside Temporal Dead Zone (TDZ).
- **this keyword:**arrow functions lack dynamic this binding, regular functions evaluate context at runtime.
- **Propagation:** preventDefault blocks default browser actions; stopPropagation blocks bubbling upwards.

💬 IMPORTANT INTERVIEW QUESTIONS & ANSWERS

Q1. Difference between var, let, and const?

Ans: var is function-scoped, hoisted as undefined, and allows re-declaration. let and const are block-scoped, hoisted into the Temporal Dead Zone (TDZ), and reject re-declaration. const also rejects re-assignment.

Q2. What is Temporal Dead Zone (TDZ)?

Ans: The period from the entry of a block scope until the point where a variable declared with let or const is initialized. Accessing variables in this zone throws a ReferenceError.

Q3. How does this behave in arrow functions?

Ans: Arrow functions do not bind their own this. They resolve the this reference lexically by inheriting it from their enclosing parent context.

Q4. Difference between == and ===?

Ans: == compares values after performing implicit type conversion (coercion). === compares values and types directly without type coercion.

Q5. Difference between null and undefined?

Ans: undefined is the default value assigned to variables that have been declared but not initialized. null is an intentional assignment representing an empty value reference.

Q6. What is a Closure?

Ans: A function that retains access to its lexical parent scope variables even after the parent function has finished executing and popped off the call stack.

Q7. Difference between map, filter, and reduce?

Ans: map transforms each array element to create a new array. filter creates a new array with elements that pass a boolean test. reduce aggregates all array elements into a single accumulator value.

Q8. What is the difference between event bubbling and event capturing?

Ans: Event bubbling propagates the event upward from the target child to the parents (default). Event capturing trickles the event downward from the parent nodes to the target child.

Q9. What is Event Delegation?

Ans: The performance pattern of placing a single event listener on a parent element to handle events for all children (existing and future dynamic ones) utilizing event bubbling.

Q10. Predict output of the following hoisting code:

javascript

```
console.log(x);

var x = 10;
```

Ans: undefined. The declaration var x is hoisted and initialized to undefined, while the value assignment x = 10 remains at its original line.

Q11. Predict output of the following TDZ code:

javascript

```
console.log(y);

let y = 10;
```

Ans: ReferenceError: Cannot access 'y' before initialization (due to the Temporal Dead Zone).

Q12. Predict output of the following Closure code:

javascript

```
function create() {
   let list = [];
   for (var i = 0; i < 3; i++) {
      list.push(() => console.log(i));
   }
   return list;
}
```

create()[0]();

Ans: 3. (Because i is declared with var, it is function-scoped and ends with value 3. All closure references point to the same variable i).

Q13. Predict output of the following this scope code:

javascript

```
const profile = {
   username: "Rohan",
   show: () => console.log(this.username)
};
profile.show();
```

Ans: undefined. (Arrow function inherits lexical this from outer global window context).

Q14. Predict output of the following Event Loop code:

javascript

```
console.log("Start");
setTimeout(() => console.log("Timeout"), 0);
Promise.resolve().then(() => console.log("Promise"));
console.log("End");
```

Ans: "Start" "End" "Promise" "Timeout" (Because Promise microtask callbacks have higher execution priority than setTimeout macrotasks).

Q15. Predict output:

javascript

```
console.log(typeof NaN);
```

Ans: "number".

Q16. Difference between Shallow Copy and Deep Copy?

Ans: Shallow copy duplicates only the top-level property references. Deep copy duplicates all nested structural levels completely, creating independent memory structures.

Q17. Difference between event.target and event.currentTarget?

Ans: e.target is the actual element that triggered the event (the click origin). e.currentTarget is the element to which the active event listener is attached.

Q18. Predict output:

javascript

```
let obj1 = { a: 1 };

let obj2 = { a: 1 };

console.log(obj1 === obj2);
```

Ans: false. (Objects are compared by their memory reference addresses, which are different for separate objects).

Q19. Predict output:

javascript

```
let x = [1, 2];

let y = [...x];

y.push(3);

console.log(x.length);
```

Ans: 2. (Spread operator creates a shallow copy of the array; modifications on the copy do not affect the original).

Q20. Explain structuredClone().

Ans: A modern built-in browser API method used to create deep copies of object trees safely.

Q21. Predict output:

javascript

```
console.log(add(2, 3));

function add(a, b) { return a + b; }
```

Ans: 5. (Standard function declarations are fully hoisted).

Q22. Predict output:

javascript

```
console.log(multiply(2, 3));

var multiply = function(a, b) { return a * b; }
```

Ans: TypeError: multiply is not a function. (The variable declaration is hoisted as undefined, and invoking undefined as a function throws TypeError).

Q23. What is the default prototype link of standard objects?

Ans: Object.prototype, which eventually links to null.

Q24. How do you create an object without a prototype?

Ans: By using Object.create(null).

Q25. Predict output:

javascript

```
console.log([] == ![]);
```

Ans: true. (Boolean check conversions evaluate array reference comparison coercion to true).

Q26. Predict output:

javascript

```
console.log(1 + "2" + 3);
```

Ans: "123". (Coerces additions to string concatenation).

Q27. Predict output:

javascript

```
console.log(1 + +"2" + 3);
```

Ans: 6. (The unary plus +"2" converts the string to number 2, resulting in 1 + 2 + 3 = 6).

Q28. What is the scope of variables declared inside loops using let?

Ans: A separate block scope is created for every loop iteration.

Q29. Difference between Debouncing and Throttling?

Ans: Debouncing triggers the function only after a certain period of inactivity (silence delay). Throttling limits the execution frequency, running the function at most once every specified time interval.

Q30. Why is response.ok used in Fetch API check?

Ans: Because fetch() does not automatically reject server status errors (404/500). response.ok checks if the status is within the success range (200-299).

10:29 PM